CS & IT ENGINEERING



Algorithms

Dynamic Programming (DP)



Lecture No.- 03

By- Aditya sir

Recap of Previous Lecture



Topic

Topic

Dijkstra's SSSP

Intro to DP

Topics to be Covered



- Topic
- Topic
- Topic

Dynamic Prog



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW



Topic : Dynamic Programming: (DP)

DP vs greedy

1. 0/1 Knapsack
2. Coin change problem
3. Multi-stage graph

Greedy → Local optimisation

DP→ Global optimisation

DP gives optimal solution but not give optimal solution.



Topic : Dynamic Programming: (DP)

Fibonacci Number:

Fib series: 0,1,1,2,3,5,8,13,21,34.

Fib(n) = Fib (n-1) + Fib (n-2), n > 1

Fib(n) =1, n = 1

Fib (n) = 0, n = 0



Topic : Dynamic Programming: (DP)

Normal Recursive Implementation of Fib (n):

```
Algo Fib (int n)
{
        If (n≤1) return (n);
        else
        {
              return (Fib (n-1)+ Fib (n-2));
        }
}
```



Topic : Dynamic Programming: (DP)

Time complexity Recurrence

$$T(n) = C,\ n = 1$$

$$T(n) = T(n-1) + T(n-2) + a\ ,\ n > 1$$

We know,

$$T(n-2) < T(n-1)$$

$$T(n) = T(n-1) + T(n-1) + a,\ \ n > 1$$

$$T(n) = 2T\ (n-1) + a\ ,\ n > 1$$

$$T(n-1) = 2T\ (n-2) + a$$



Topic : Dynamic Programming: (DP)

$T(n) = 2[2T(n-2) + a] + a$

$= 2^2 T(n-2) + 2a + a \ldots\ldots(2)$

$T(n-2) = 2T(n-3) + a$

$T(n) = 2^2 (2T(n-3) + a) + 2a + a$

$= 2^3 T(n-3) + (2^2 + a\ 2^1 a + 2^0 a)$

General Term

$T(n) = 2^k T(n-k) + (2^k - 1) * a$

$n - k = 1 \rightarrow k = (n-1)$



Topic : Dynamic Programming: (DP)

$T(n) = 2^{(n-1)} * T(1) + (2^{(n-1)} - 1) * a$

$$= c * \frac{2^n}{2} + \left(\frac{2^n}{2} - 1\right) * a$$

$T(n) \rightarrow O(2^n)$



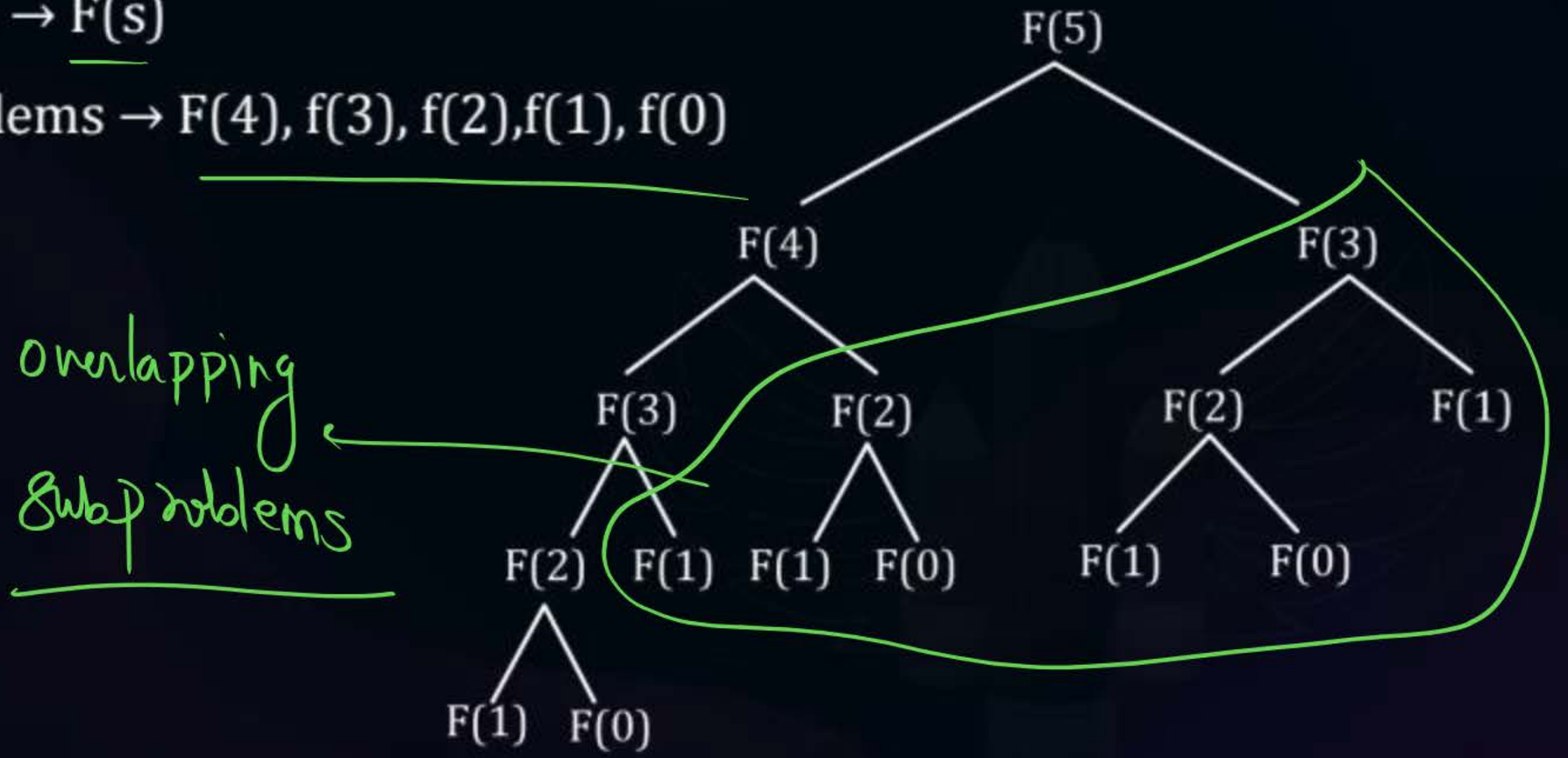
Topic : Dynamic Programming: (DP)

Normal Recursive Approach: Tree of function calls

Original problem → F(s)

unique Sub-problems → F(4), f(3), f(2),f(1), f(0)

(Q) Find fibo (5):



overlapping Subproblems



Topic : Dynamic Programming: (DP)

Top-down- Memoized implantation of Fib (n)

```
Algo memiFb(n)
{
        if (m [n] is undefined)
        {
            if (n <=1) result = n;
            else
            result = memFib (n-1) + memFib(n-2);
            m[n] = result; //memorizing (caching)
        }
        return (m[n]);
}
```



Topic : Dynamic Programming: (DP)

Top-Down code walkthrough:

#Q. Fib (5) = ?

$0 \rightarrow 5$

	0	1	2	3	4	5
M:	0	1	1	2	3	5

?

memFib(5): res: 3 + 2 = 5→ m [5]= 5

memFib(4):res: 2 + 1 = 3→ m [4]= 3

memFib(3):res: 1 + 1 = 2→ m [3]= 2

memFib(2): res: 1 + 0 = 1→ m [2]= 1

memFib(1):m [1]= 1

memFib(0): m [0]= 0

memFib(0): 1

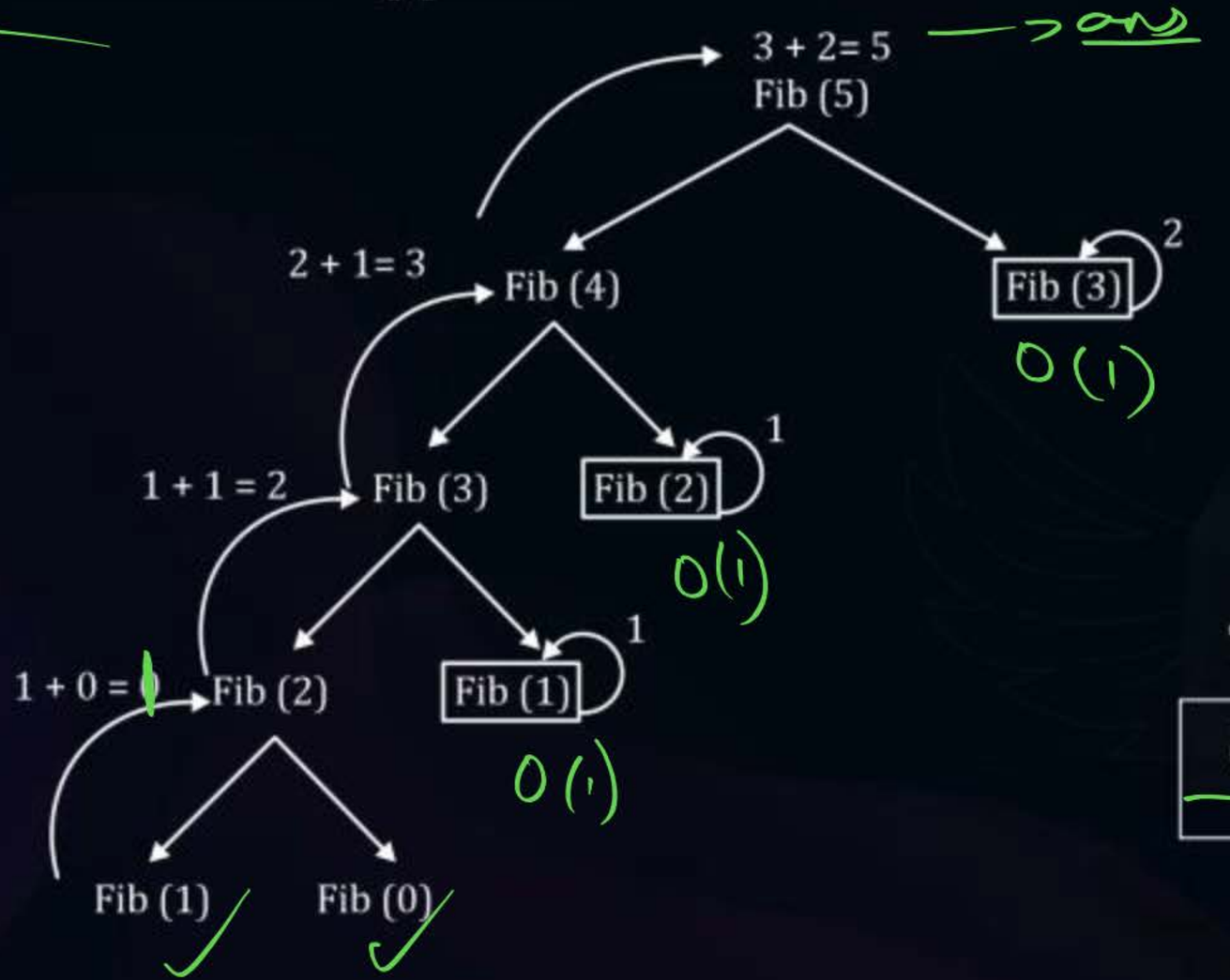
memFib(2): m[2]

memFib(3): m [3]



Topic : Dynamic Programming: (DP)

Function call Tree for above approach.



TC → O(n)

$O(2^n) \rightarrow O(n)$



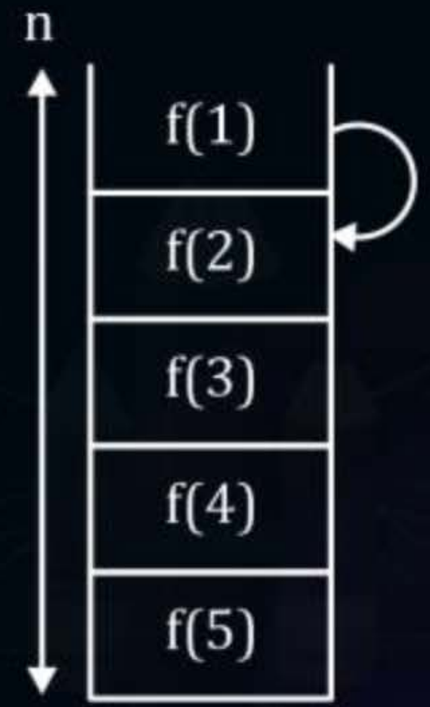
Topic : Dynamic Programming: (DP)

Space Complexity

- Recursion stock → O(n) {Stack size}
- Auxiliary array → O(n) → m[n]

Overall space → O(n + n) = O(n)

Complexity



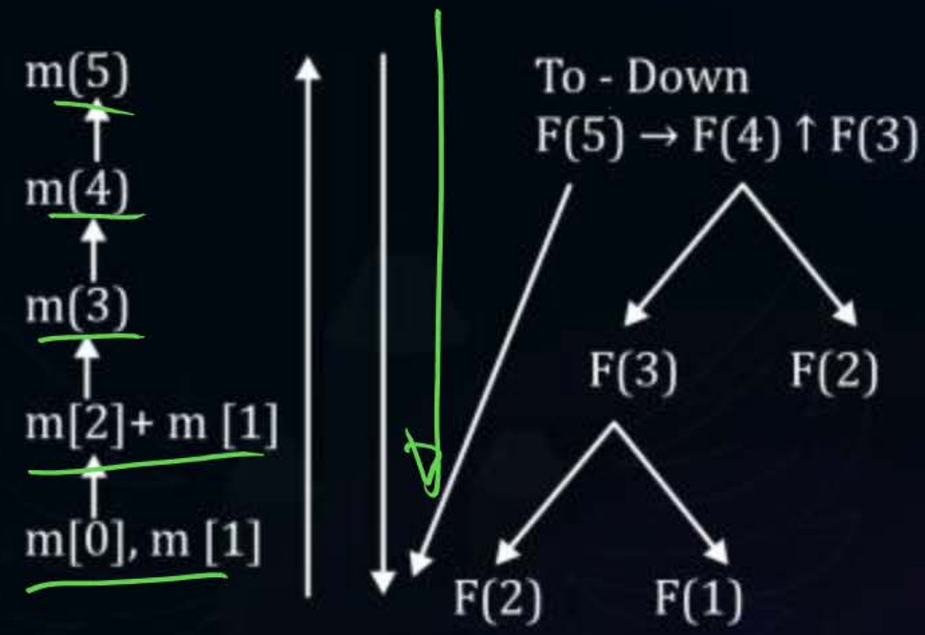


Topic : Dynamic Programming: (DP)

Bottom-up approach of D.P. for Fib (n)

```
Algo memFib (n)
{
    M[0] = 0
    M[1] = 1
    For i = 2 to n
    {
        M[i] = M [i-1] + M[i-2]
    }
    return M[n];
}
```

Appr 2: Tabulation (Iterative)





Topic : Dynamic Programming: (DP)

Eg.1. Fib(5)

Working

m

0	1	2	3	4	5
0	1	1	2	3	5

i =2 , m [2] = m [1] + m [0] = 1 + 0 = 1

i =3 , m [3] = m [2] + m [1] = 1 + 1 = 2

i =4 , m [4] = m [3] + m [2] = 2 + 1 = 3

m[5] = 3+2=5

$O(2^n) \rightarrow O(n)$

TC → O(n)

SC → O(n)



Topic : Dynamic Programming: (DP)

Dynamic Programming vs Greedy Method vs Divide & Conquer:

- In all methods the problem is divided into subproblem;
- Greedy Method: Building up of the solution to the problem is done in step-wise manner (incrementally) by applying local options only (local optimality).
- Divide & conquer: Breaking up a problem into separate problems (independent), then solve each subproblem separately (i.e. independently) & combine the solution of subproblems to get the solution of original problem.
- Dynamic Programming: Breaking up of a problem into a series of overlapping subproblems & building up solution of larger & larger subproblems.



Topic : Dynamic Programming: (DP)

- Unlike Divide and Conquer, D.P typically involves solving all subproblems, rather than a small portion of subproblem.
- D.P tends to solve each subproblem only once since the results of the subproblems are stored, which are used later again when required. This is going to reduce the computation drastically. (In most case, the complexity)

Ex- Fib
- Brute force: O(2)
- D.P Implementation: O(n)



Topic : Dynamic Programming: (DP)

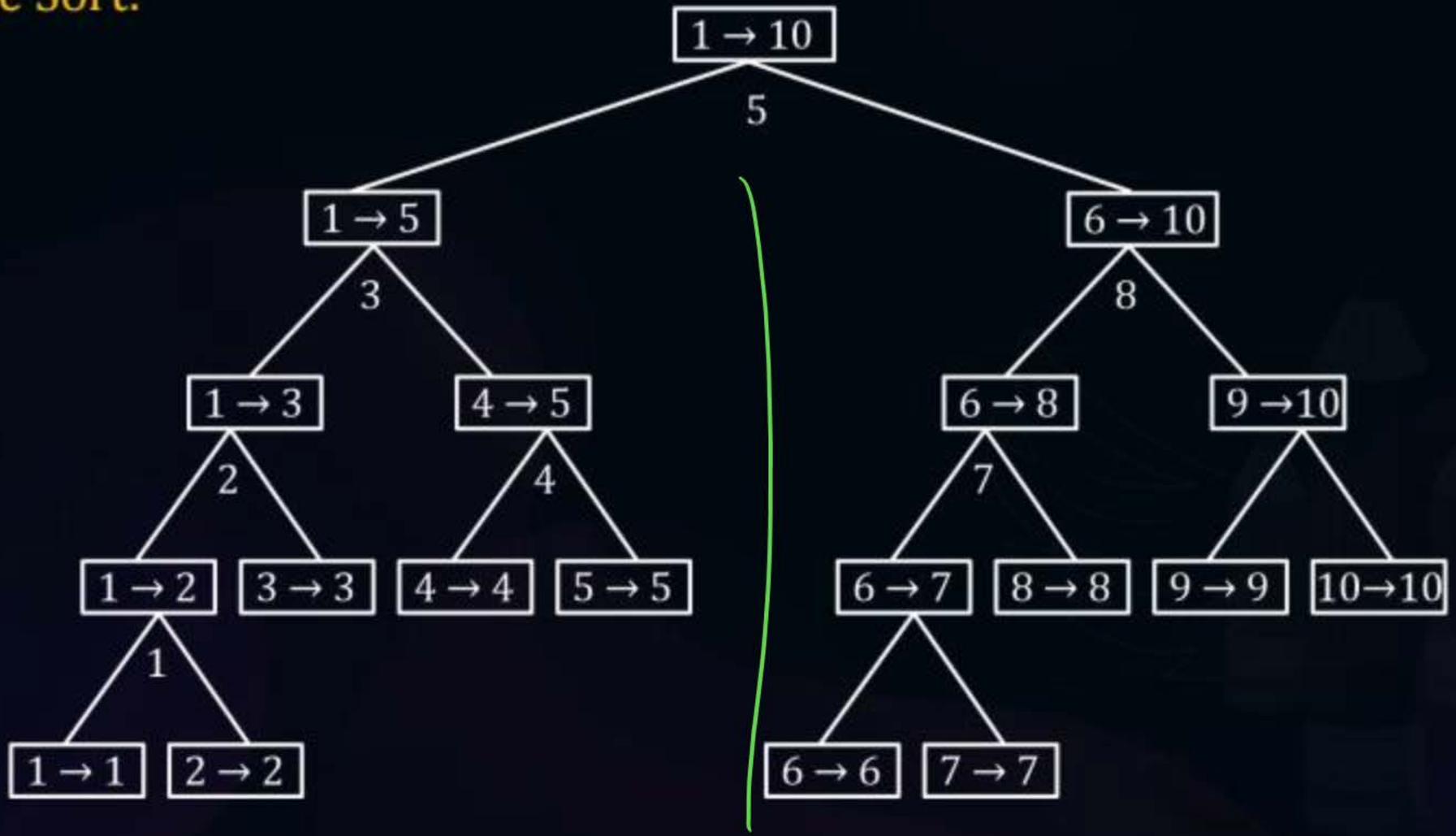
Imp. Diff. D_nC and DP:

1. DnC: independent sub-problems some separately
2. DP: Overlapping Sub- problems



Topic : Dynamic Programming: (DP)

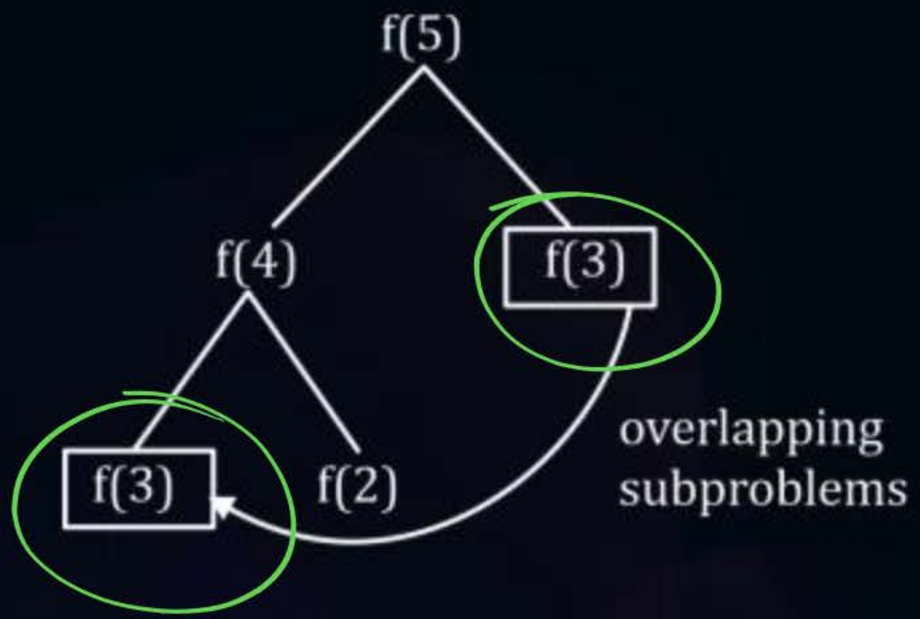
Merge Sort:





Topic : Dynamic Programming: (DP)

DP → Fibonacci



Application of DP:



Topic : Dynamic Programming: (DP)

Single source Shortest path (SSSP)

1. Dijkstra → Greedy
2. Bellman ford → DP

Distinction between both SSSP algorithms and which to use when?



Topic : Dynamic Programming: (DP)

1. Dijkstra's SSSP (greedy algo) always gives optimal solution to the SSSP problems, provided all the edges in the given graph are of +ve weight.

2. If the graph has any –ve weight edge, then Dijkastra SSSP algo may or May not give optimal solution. (Path costs)



Topic : Dynamic Programming: (DP)

3. If the graph has 0 or more –ve weight edge But NOT a –ve weight cycle, then the bellman ford SSSP algo (DP based) always gives optimal solution to SSSP problem.

4. If the graph has any –ve weight cycle that is reachable from source, then No algorithm works.



Topic : Dynamic Programming: (DP)

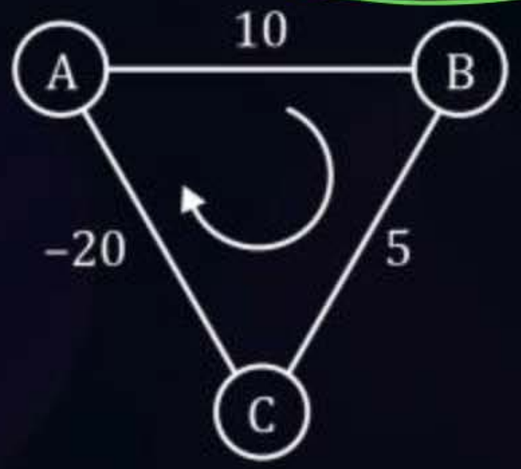
-ve weight cycle?

Net weight of cycle = 10 + 5 – 20

= – 5

(-ve weight cycle)

* Min with path is not defined.



A → B?

10?

A → B: 10

A → B → C → A → B

- 5 + 10 = 5

10 → 5 → 0 → – 5 → –10

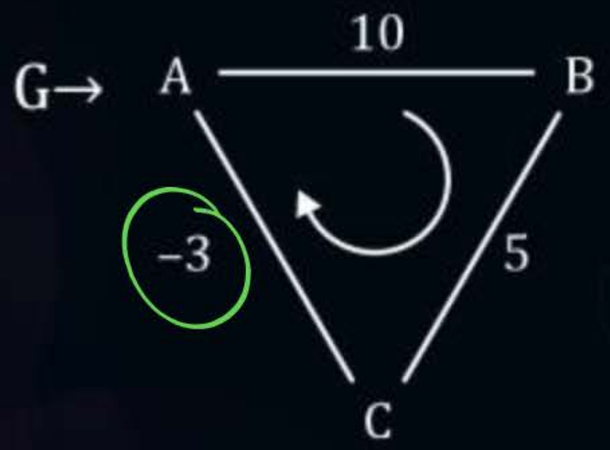
Topic : Dynamic Programming: (DP)

Eg.1. Graph with -ve weight edge but No required with cycle.

-ve wt

Net weight = 10 + 5 – 3

= 12

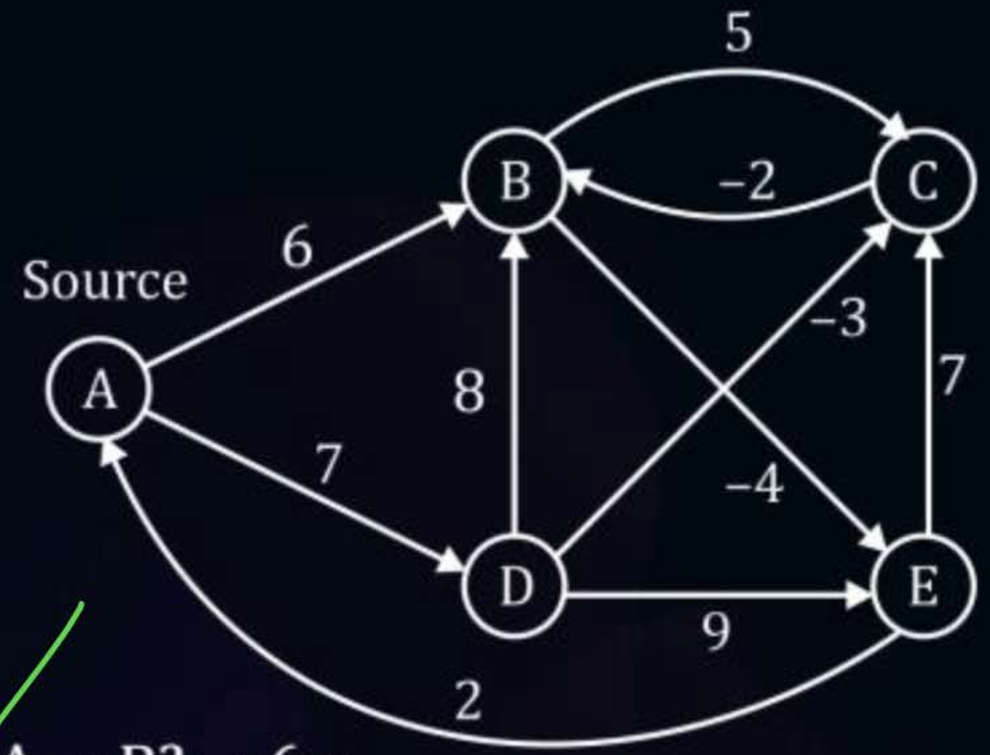


BF ✓

Dijkstra SSSP→ Not guarantee (-ve weight edge)

Topic : Dynamic Programming: (DP)

Imp Given: G



$A \to B? \to 6 \times$

$A \to D \to C \to B \Rightarrow 7 + (-3) + (-2)$

$= 7 - 5 = 2$ {Actual Minimum}

Apply Dijkstra SSSP Algo



	A	B	C	D	E
A→	0	6	4	7	2
		×	✓	✓	×

$A \to E : 2$

$A \to D \to C \to B \to E \Rightarrow 7 + (-3) + (-2) + (-4)$

$= 7 - 9 = -2$ {Actual min}



Topic : Dynamic Programming: (DP)

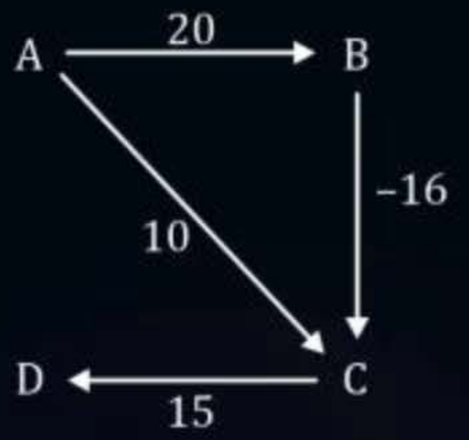
IMP Note:

1. In previous e.g. Dijkastra SSSP failed to give optimal path cost to few vertices, because in Dijkstra, once the vertex is selected it is not further considered to be relaxed.

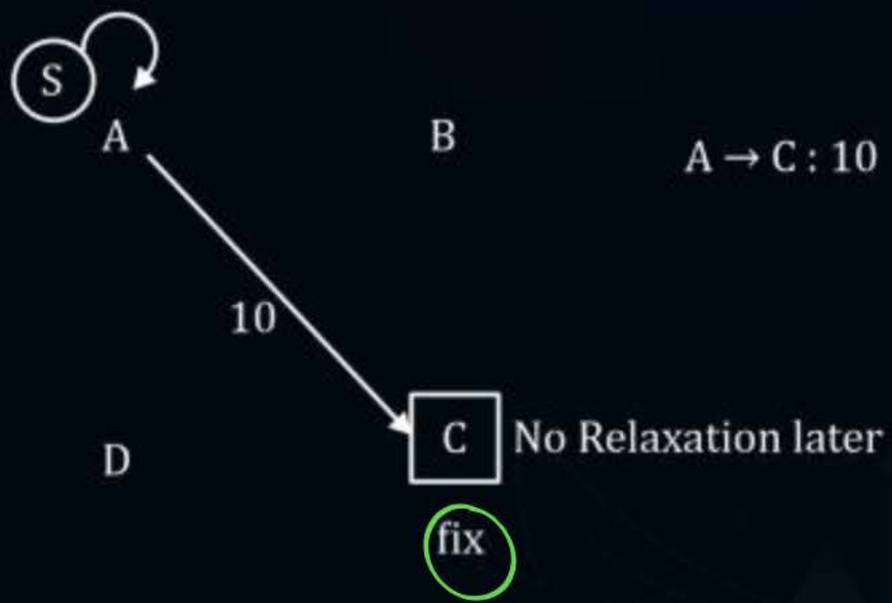
2. In Bellman ford, the relaxation is carried out w.r.t. to the edges in multiple iterations (n-1) iterations.

Topic : Dynamic Programming: (DP)

Idea:-

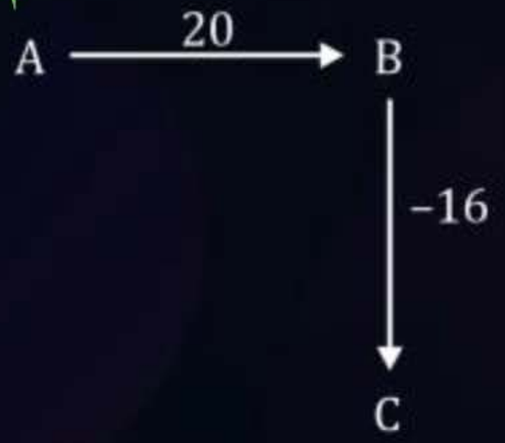


Dijkstra SSSP



$A \rightarrow C : 10$

Actual



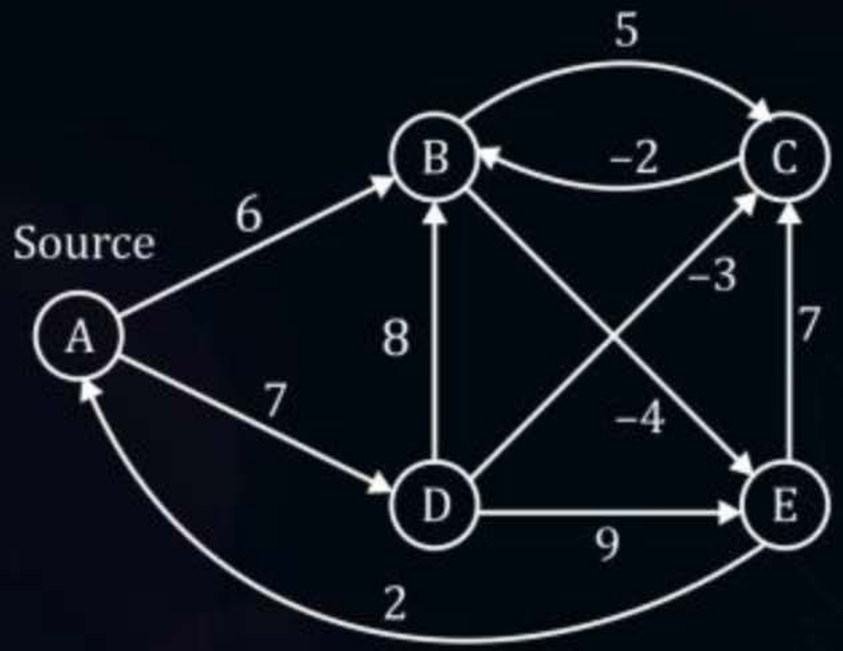
$\Rightarrow A \rightarrow C : \quad 20 - 16$

$= 4$



Topic : Dynamic Programming: (DP)

BF



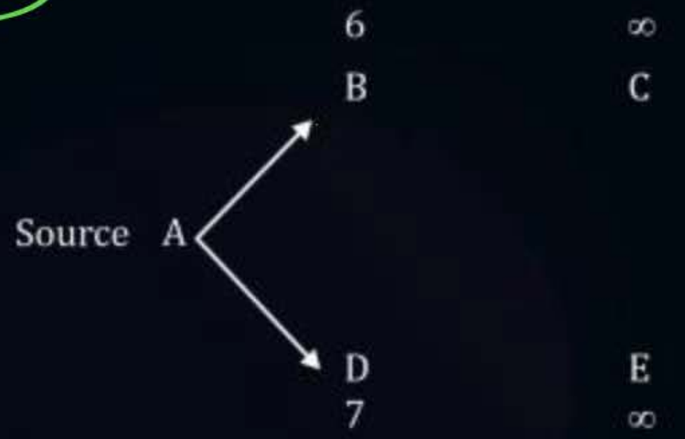
n = 5

Iteration: n −1 = 4

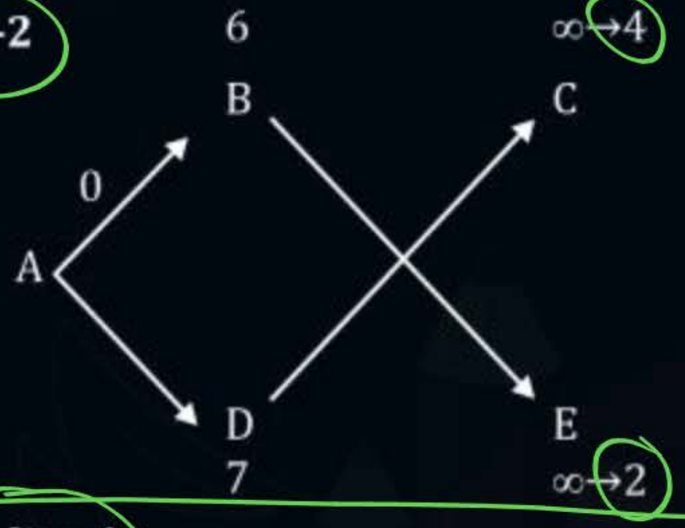
Topic : Dynamic Programming: (DP)

Bellman ford Walk through

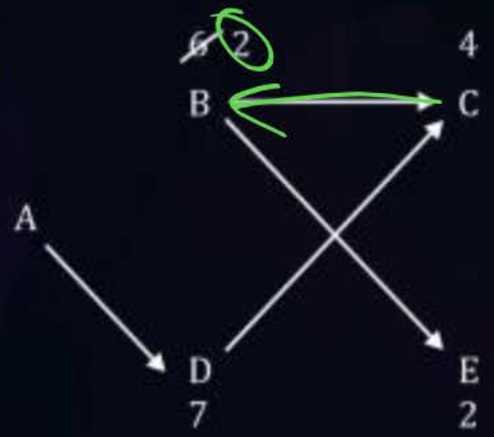
Iter.-1



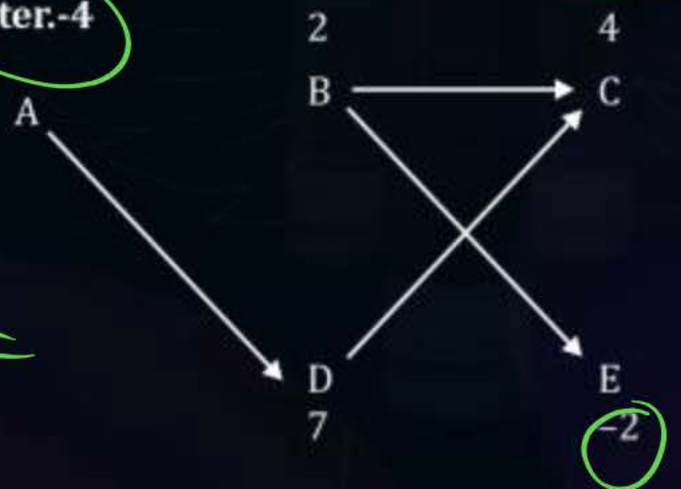
Iter.-2



Iter.-3



Iter.-4



final ans by BF



Topic : Dynamic Programming: (DP)

Dj →		
	A → B : 6 ✗	A → B : 2
	A → C : 4	A → C : 4
	A → D : 7	A → D : 7
	A → E : 2 ✗	A → E : −2

Imp. Observation:

Dijkastra Greedy SSSP algorithm failed to give us the shortest path to few vertices. But bellman ford SSSP given shortest paths (optimal solution) to all the vertices. (Even through −ve weight edge are present but no-ve weight cycle.)



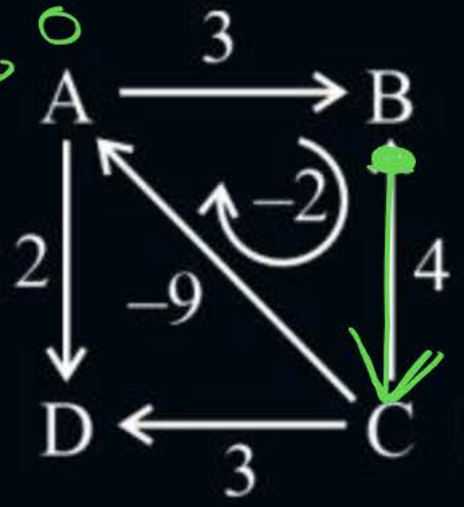
Topic : Dynamic Programming

Bellman Ford:

$n = 4$

$n - 1 = 3$

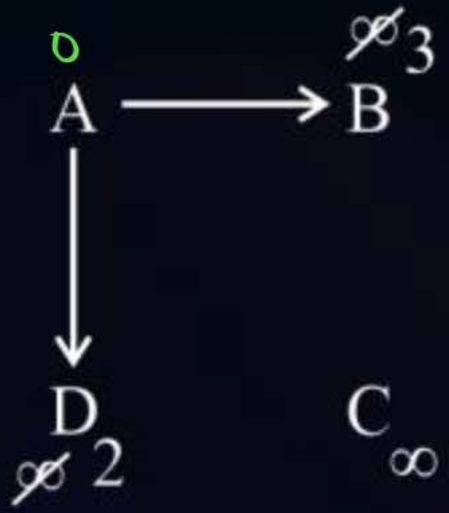
Source



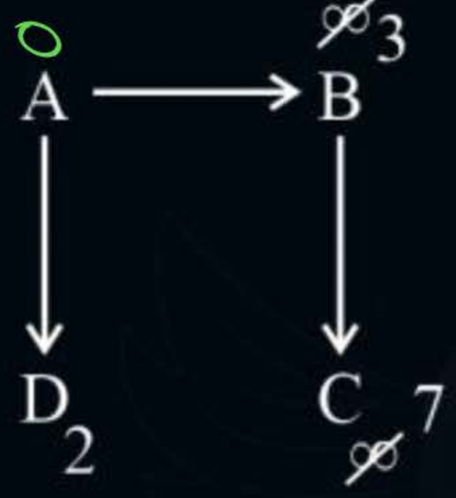


Topic : Dynamic Programming

Iteration 1:



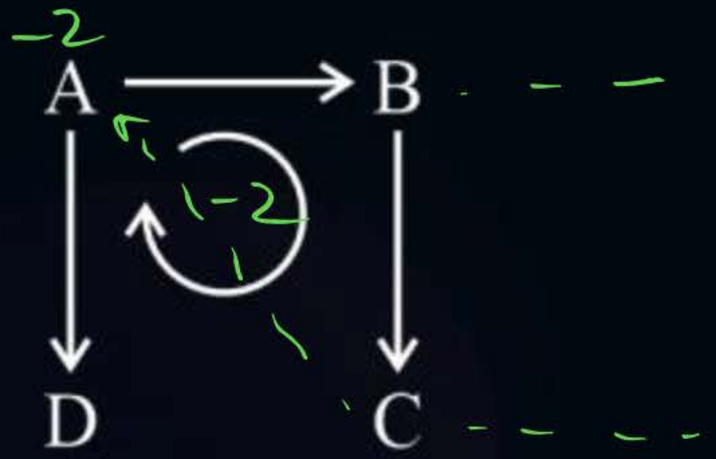
Iteration 2:





Topic : Dynamic Programming

Iteration 3:



In above example, as there is a negative wt cycle reachable from source, so even Bellman Ford fails



Topic : Dynamic Programming

Algorithm Bellman-Fort (G, w, s)

```
1.  Initialize-Single-Source(G,s)
2.  for i ← 1 to |V[G]| – 1
3.      do for each edge (u, v) ∈ E[G]
4.          do Relax(u, v, w)
5.  for each edge (u, v) ∈ E[G]
6.      do if d[v] > d[u] + w(u, v)
7.          then return FALSE
8.  return TRUE
```

→ (BF does not work → -ve wt cycle)



Topic : Dynamic Programming

Initialize – Single- Source (G, s)

1. For each vertex v ∈ V[G]
2. do d [u] ← ∞
3. π[v] ← NIL
4. d[s]← 0



Relax (u, v, w) → adj matrix

1. if d[v] > d[u] + w(u, v)
2. then d[v] = d[u] + w[u,v]
3. π[v] ← u



Topic : Dynamic Programming

Overall Time Complexity:

- Bellman Ford SSSP DP approach

  TC → O(n + n*e + e) = O(n * e)

[NAT]

P.49

#Q. The time complexity of Bellman Ford algo for a complete graph having n vertices is?

TC: $O(n * e)$

Complete graph: $e = O(n^2)$

TC: $O(n * n^2)$

TC: $O(n^3)$

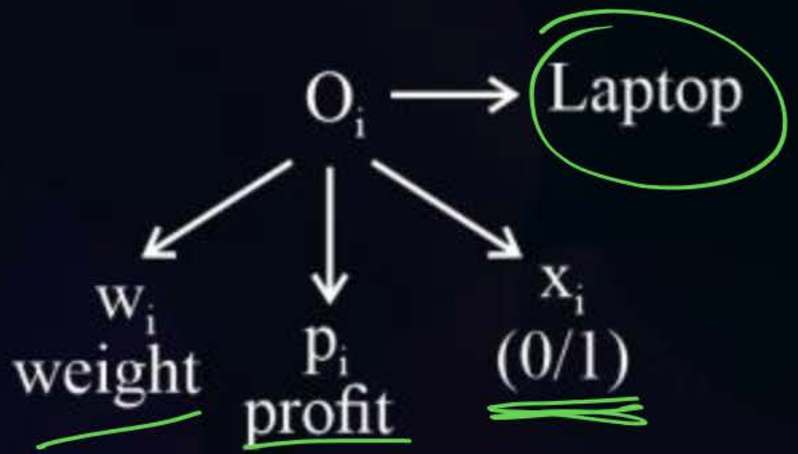
0/1 Knapsack



Topic : Dynamic Programming

0/1 Knapsack (Binary Knapsack):

- Knapsack Capacity : M
- no. of objects → n





Topic : Dynamic Programming

Objective Function:

$$\max \sum_{i=1}^{n} p_i * x_i$$

Such that:

$$\Sigma w_i * x_i \leq M$$

$x_i = \{0, 1\}$

$0 \leq x_i \leq 1$

↓

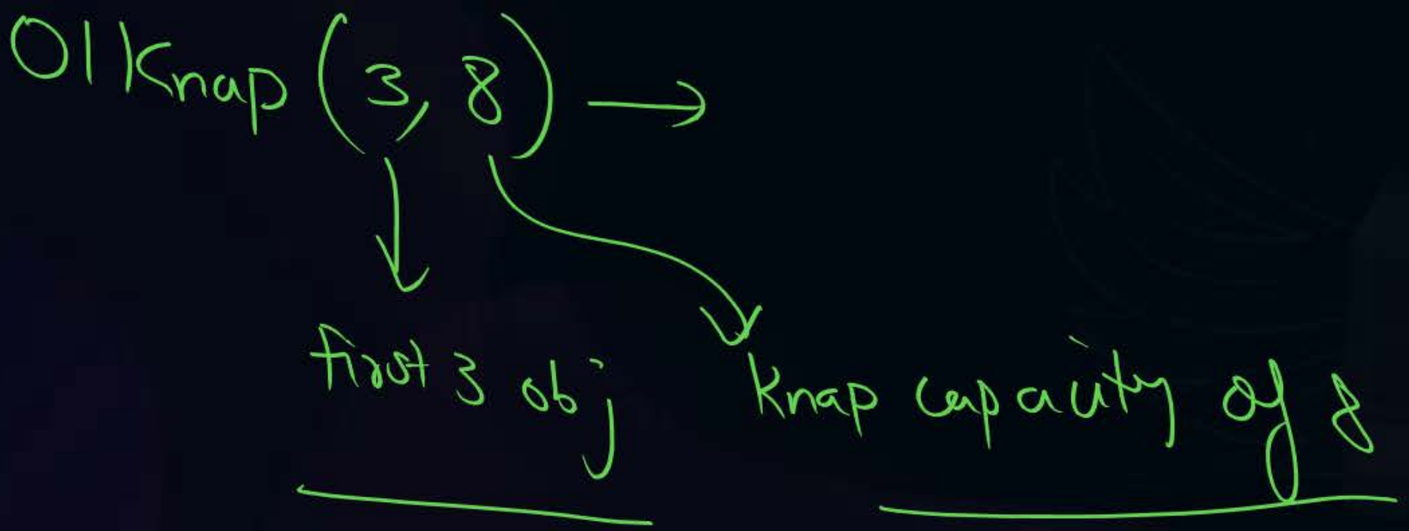
Fractional



Topic : Dynamic Programming

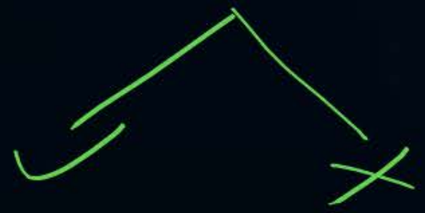
Dynamic Programming Approach:

- Let 01 Knap(n, M) represent max profit with n-objects and Knapsack of Capacity M.





Topic : Dynamic Programming



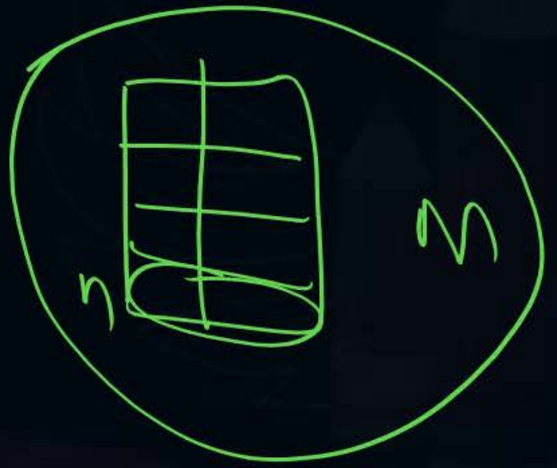
Recurrence: (Top - Down)

$$01\ Knap(n, M) = 01\ Knap(n-1, M),\ w_n > M \quad \rightarrow \text{Exclude } O_n \text{ (no option)}$$

$$01\ Knap(n, M) = \max \begin{cases} 01\ Knap(n-1, M) & \text{excluding} \\ 01\ Knap(n-1, M-w_n) + p_n & \text{including} \end{cases}, \quad \text{if } w_n \le M$$

Initialization,

$$01\ Knap(n, M) = 0, \text{ if } n = 0 \text{ or } M = 0$$





Topic : Dynamic Programming

Example:

To understand if there are overlapping Sub-problem or not?

n = 4, M = 3

Yes

	01	02	03	04
P_i profits	10	20	30	40
W_i weights	1	1	1	1



Topic : Dynamic Programming

Top-Down (Recursive):

overlapping subprob ✓





Topic : Dynamic Programming

Bottom-up Approach (Tabulation):

Example:

n = 4, M = 8

$(n+1) * (M+1)$

	01	02	03	04
P_i	1	2	3	6
W_i	2	3	4	5

Tabulation:

Let X[0...n, 0...M] be an ~~array~~ matrix of which X[n, M] ⇒ final answer (max profit)



Topic : Dynamic Programming

P_i	W_i
1	2
2	3
5	4
6	5

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0	0	1	2	2	3	3	3	3
3	0	0	1	2	5	5	6	7	7
4	0	0	1	2	5	6	6	7	8

m

m

n

m[0, n] } 0
m[n, 0] }



Topic : Dynamic Programming

exclude obj i

j

$X[i, j] = \max\{X[i-1, j], X[i-1, j-W_i] + P_i\}$

include obj i

$X[2, 3] = \max\{X[1, 3], X[1, 3-3] + 2\}$

$X[2, 3] = \max\{1, 0+2\} = \max[1, 2] = 2$

X[2, 5] = max profit we can have with considering only 1st two objects and knapsack of capacity 5.



2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

DP

↓

SSSP , PYQ

0/1 Knap



THANK - YOU